

Reviewer Pack — Minimal Rank of Braid Monodromy Representations over XOR-AND...

Entry 4e0b8a57f448 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 21:48:04 UTC

Minimal Rank of Braid Monodromy Representations over XOR-AND Tree Width

Entry ID: 4e0b8a57f448 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 21:48:04 UTC

1. Conjecture

Field A (mathematical branch): Braid Theory **Field B** (complexity object): Complexity Theory: XOR-AND Tree Width

Statement:

[‘For any XOR-AND tree T with n leaves, the minimal rank of the braid monodromy representation associated with T is $\Theta(\log^2(n))$.’, ‘Equivalently, for any instance of k -CLIQUE on n variables, there exists a braid monodromy representation whose associated matrix has at most $\Theta(k \log^2(n))$ non-zero entries.’, ‘Furthermore, the minimal rank of this matrix is upper bounded by $k \log^2(n)$.’]

Rationale (proposer’s reasoning):

[‘Braid theory provides a rich algebraic structure that can be used to encode complexity-theoretic objects like XOR-AND trees and k -CLIQUE instances. The conjecture suggests that this encoding leads to efficient representations in terms of braid monodromy representations, which could have implications for monotone circuit lower bounds.’, ‘Braid monodromy representations are known to be closely related to the Jones polynomial, a classical invariant in knot theory. The conjecture proposes an extension of these relationships to XOR-AND trees and k -CLIQUE instances.’, ‘If true, this conjecture would provide a novel approach for studying monotone circuit lower bounds using the tools of braid theory.’]

Taxonomy category: MONOTONE_CLIQUE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3f9eedb1dc1e5404

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean minimal rank of braid monodromy representations across multiple seeds is within 10% of $\log^2(n)$ for XOR-AND trees, and for k -CLIQUE instances, the mean minimal rank does not exceed $k \log^2(n)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - braid theory AND XOR-AND tree width - braid monodromy representation AND complexity theory XOR-AND tree width - $\Theta(\log^2(n))$ minimal rank braid monodromy AND XOR-AND tree

Top relevant hits considered: - [<http://arxiv.org/abs/2106.13355v2>] Cohomology ring of tree braid groups and exterior face rings - [<http://arxiv.org/abs/2510.04248v1>] Trisected Rainbows and Braids - [<http://arxiv.org/abs/2209.03860v2>] Graph of groups decompositions of graph braid groups - [<http://arxiv.org/abs/1603.02493v4>] Clifford theory for glider representations - [<http://arxiv.org/abs/0711.0795v4>] Finite-Dimensional Representations of Hyper Loop Algebras Over Non-Algebraically Closed Fields - [<http://arxiv.org/abs/1101.2456v3>] Polynomial representations and categorifications of Fock Space - [<http://arxiv.org/abs/2510.17487v1>] Directional Search for Persistent Gravitational Waves: Results from the First Part of LIGO-Virgo-KAGRA's Fourth Observin - [<http://arxiv.org/abs/1411.4413v2>] Observation of the rare $B_s^0 \rightarrow \mu^+ \mu^-$ decay from the combined analysis of CMS and LHCb data

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    k = random.randint(1, min(n-1, 10))

    # Generate a random XOR-AND tree with n leaves
    def generate_xor_and_tree(n):
        if n == 1:
            return [0]
        left = generate_xor_and_tree(n // 2)
        right = generate_xor_and_tree(n - n // 2)
        return [left, right]

    tree = generate_xor_and_tree(n)

    # Compute the braid monodromy representation for XOR-AND trees
```

```

def xor_and_braid_monodromy(tree):
    if isinstance(tree, int):
        return [[1]]
    left = xor_and_braid_monodromy(tree[0])
    right = xor_and_braid_monodromy(tree[1])
    result = []
    for l in left:
        for r in right:
            result.append([l[i] + r[i] for i in range(len(l))])
    return result

braid_rep = xor_and_braid_monodromy(tree)

# Compute the minimal rank of the braid monodromy representation
def matrix_rank(matrix):
    rows, cols = len(matrix), len(matrix[0])
    if rows == 0 or cols == 0:
        return 0

    # Gaussian elimination to find the rank
    for i in range(rows):
        max_row = i
        for j in range(i+1, rows):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j

        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        if matrix[i][i] == 0:
            continue

        for j in range(i+1, rows):
            factor = -matrix[j][i] / matrix[i][i]
            for k in range(cols):
                matrix[j][k] += factor * matrix[i][k]

    rank = sum(1 for row in matrix if any(row))
    return rank

minimal_rank = matrix_rank(braid_rep)

# Check the conjecture
expected_minimal_rank = math.log2(n) ** 2
conjecture_holds = abs(minimal_rank - expected_minimal_rank) / expected_minimal_rank <= 0.1

return {
    "metric_name": "minimal_rank",
    "metric_value": minimal_rank,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"n={n}, expected_minimal_rank={expected_minimal_rank}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 97) for _ in range(30)]

```

```

results = []
for seed in seeds:
    trial_result = run_trial(seed)
    print(f"TRIAL: {trial_result}")
    results.append(trial_result)

mean_rank = sum(result["metric_value"] for result in results) / len(results)
std_rank = math.sqrt(sum((result["metric_value"] - mean_rank) ** 2 for result in results) / len(results))
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_rank:.2f} std={std_rank:.2f} support_fraction={support_fraction:.2f}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_rank:.2f} std={std_rank:.2f} support_fraction={support_fraction:.2f}")
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7258f710.py", line 93, in <module>
    trial_result = run_trial(seed)
                    ^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7258f710.py", line 45, in run_trial
    braid_rep = xor_and_braid_monodromy(tree)
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7258f710.py", line 37, in xor_and_braid_monodromy
    left = xor_and_braid_monodromy(tree[0])
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7258f710.py", line 37, in xor_and_braid_monodromy
    left = xor_and_braid_monodromy(tree[0])
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7258f710.py", line 37, in xor_and_braid_monodromy
    left = xor_and_braid_monodromy(tree[0])
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

[Previous line repeated 2 more times]

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7258f710.py", line 38, in xor_and_braid_monodromy
    right = xor_and_braid_monodromy(tree[1])
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the pre-registered support conditions could not be evaluated. | next: Investigate and fix the crash in the test code to allow for proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12591
2	propose	ollama_remote	glm4:latest	0	0	9761
3	propose	ollama_remote	glm4:latest	0	0	10956
4	preregistration	ollama_remote	glm4:latest	0	0	6181
5	novelty	ollama_remote	glm4:latest	0	0	4792
6	novelty	ollama_remote	glm4:latest	0	0	6219
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19679
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10875
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8499
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10860
11	judge	ollama_remote	glm4:latest	0	0	8532

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 108945 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in research/audit/4e0b8a57f448.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/4e0b8a57f448.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/4e0b8a57f448.tar.gz (if generated)